Metro Navigator — Technical Design

Algorithms, data formats, constants, threading, tests and tuning

25 September 2026

Document control

Item	Value
JIRA	**NH-3779**
Document	Technical Design, version 2.1 — Metro Navigator (`:app`) only
Date	25 September 2026
Companion	`01_High_Level_Design.md`, `03_Technology_Guide.md`
Code state	82 JVM tests passing, lint 0 errors, `assembleDebug` OK

1. Source layout (`app/src/main/java/com/example/testgpslocation`)

File	Role
`MainActivity.kt`	Screen: status, position, line/stations, destination, warnings with fixes, history list, permissions
`service/LocationTrackerService.kt`	Foreground service (type `location`), wake lock, writes positions/events to the daily log
`engine/NavigationController.kt`	Process-wide owner: `SensorHub`, `RadioScanner`, `UndergroundNavigator`, `MetroRouteService`, radio-map persistence; exposes `StateFlow`s
`engine/UndergroundNavigator.kt`	**Positioning engine** (pure Kotlin)
`engine/TrackHmm.kt`	Hidden Markov model over positions along one line
`engine/RadioMap.kt`	Self-learnt Wi-Fi/BLE fingerprint database, anchors, radio fix, text persistence
`engine/Pdr.kt`	Step length and phone heading
`engine/RideMemory.kt`	Per-run moving time → run cruise speed; `ride_memory.txt`
`engine/MetroRoute.kt`	Immutable line: polyline, cumulative distances, stations, projection; `TrackStitcher`
`engine/MetroRouteService.kt`	Bundled-asset import, phone cache, Overpass download
`engine/OverpassParser.kt`	Overpass JSON → `MetroRoute` (shared by bundle and download)
`sensor/SensorHub.kt`	GNSS, fused/network location, IMU windows, step detector, barometer
`sensor/RadioScanner.kt`	Wi-Fi scans, BLE beacon scan, Wi-Fi RTT

File	Role
`sensor/BeaconParser.kt`	iBeacon / AltBeacon / Eddystone-UID identity from raw advertisements
`app/src/main/assets/metro/*.json.gz`	Bundled metro lines
`scripts/build_metro_bundle.py`	Generates the bundle (build time)

Library `:core`: `GpsPoint`, `GeoToolbox` (haversine, bearing, destination point), `NetworkStatus`, `TelemetryLogger` (daily MediaStore / file log).

2. Build, run, test

```
./gradlew :app:assembleDebug                # APK: app/build/outputs/apk/debug/app-debug.apk
./gradlew :app:testDebugUnitTest            # 82 JVM tests incl. ride simulations
./gradlew :app:lintDebug                    # 0 errors
python3 scripts/build_metro_bundle.py       # refresh bundled metro lines (needs internet, bui
python3 scripts/build_metro_bundle.py pune  # one city
```

Test-only dependency: `org.json:json` (Android's `org.json` is a stub in JVM tests), used to parse the real bundled files in `BundledLinesTest`.

3. Runtime structure and threading

- All engine state is touched **only on the main thread**: `SensorHub` and `RadioScanner` deliver on the main looper; BLE and RTT callbacks run on the main executor.
- `NavigationController` publishes `NavState` (≤ 4 Hz), positions (`SharedFlow`, every GPS fix ≥ 1 s apart, every 2 s without GPS), messages and line status.
- Background work (`Dispatchers.IO`): bundle import, cache load/save, Overpass download, radio-map file read/write. Radio map text is built on main (plain `StringBuilder`, no formatting) and written on IO with temp file + rename.
- Lifecycle: `acquire(client)` starts `SensorHub` then `RadioScanner`; when the last client releases, both stop and the radio map is saved.

4. Sensor front ends

4.1 `SensorHub`

- **GNSS:** `LocationManager.GPS_PROVIDER`, 1 s → `GpsPoint(isGnss = true)`.
- **Network:** fused location (Play services; `PRIORITY_HIGH_ACCURACY`, 2 s) or the platform network provider → `isGnss = false`. Without a GNSS receiver, fused fixes ≤ 25 m count as GNSS.
- **GPS lost:** watchdog, 6 s without a GNSS fix.
- **Motion windows (100 ms):** world-frame (East, North) linear acceleration via the rotation vector; classification STATIONARY / WALKING / MOVING from accelerometer spread (2 s ring), gyroscope mean and step events; `stationaryForMs`.
- **New fields in `MotionSample`:** `steps` (null if no step detector / permission), `stepCadenceHz` (steps in the last 3 s), `deviceHeadingDeg` (`Pdr.headingFromRotation`), `orientationKnown` (rotation vector available

→ enables inertial speed, odometer and stop/departure verification), `yawRateDps` (gyroscope rate about world "up", i.e. projected on gravity — magnetometer independent; clockwise positive).

4.2 `RadioScanner`

Part	Mechanism	Output
Wi-Fi	`startScan()` every 30 s + `SCAN_RESULTS_AVAILABLE_ACTION` receiver; results older than 10 s dropped (throttled requests return stale lists); same scan never delivered twice	`RadioScan(signals = wifi:<bssid> + current beacons)`
BLE	`BluetoothLeScanner`, `SCAN_MODE_LOW_LATENCY`, report delay 0; `BeaconParser` keeps only iBeacon (`ble:ib:<uuid>:<major>:<minor>`), AltBeacon (`ble:alt:<id>`), Eddystone-UID (`ble:eddy:<ns+inst>`); loudest RSSI per beacon in the last 3 s	`RadioScan` (BLE only) every 2 s
RTT	Android 9+, `FEATURE_WIFI_RTT`; 802.11mc responders from the last scan (≤ 60 s old) whose position is known (`isKnownTransmitter`), up to `RangingRequest.getMaxPeers()`, one request at a time	`RadioScan(ranges = [RadioRange(key, m, σ ≥ 1 m)])`

Permissions: Wi-Fi results need fine location; BLE needs `BLUETOOTH_SCAN` (12+) or location (≤ 11); RTT needs `NEARBY_WIFI_DEVICES` (13+). Each part silently disables itself if missing, and BLE restarts automatically when Bluetooth is switched on.

5. Metro line data

5.1 Sources and order

0. **Automatic current-city download (primary, worldwide)** — `NavigationController.maybeLookupLines` runs on every position fix (throttled to 10 s / 200 m); if the spot is not inside a downloaded area it calls `startDownload(radius = CITY_RADIUS_METERS = 30 km)`. On `Failed` it immediately downloads `SEARCH_RADIUS_METERS = 2 km` and retries the city with back-off; on `NoNetwork` the network callback retries as soon as a network appears. Areas expire after 7 days (silent refresh when online).
1. **Bundle** — `assets/metro/<city>.json.gz`. `MetroRouteService.preload()` imports each file once per install/update (marker file `bundled_imported.txt` keyed by `name@lastUpdateTime`), adds lines not already cached, and records the city circle as "downloaded" so no network attempt is made for 7 days. A corrupt bundle file is logged and skipped.
2. **Cache** — `files/metro_routes/<id>.json` (format version 2), atomic writes.
3. **Overpass** — used by 0 and by the manual buttons: 3 servers in turn, 70 s (area) / 190 s (city) timeouts, exponential back-off 30 s → 10 min, immediate retry when a network appears.

5.2 Bundle generation (`scripts/build_metro_bundle.py`)

Query per city (same as the app's city query): `rel["route"~"^(subway|light_rail|monorail)$"](around:R,lat,lon)->.r;.r out geom;node(r.r)["name"];out body;` The response is trimmed to the fields `OverpassParser` reads (`name`, `name:en`, `ref`, `from`, `to`, `route`, `railway`, `public_transport`, member geometry), coordinates rounded to 1e-6° (~10 cm), gzip level 9, plus a `bundle` object (city centre and radius).

Bundled at the time of writing (verified by `BundledLinesTest` with the app's parser):

City file	Lines (per direction)	Track km	Stations
delhi_ncr	32	1049	654
mumbai	16	288	216
bengaluru	10	309	275
chennai	12	386	140
kolkata	10	143	116
hyderabad	6	129	120
kochi	2	56	50
ahmedabad_gandhinagar	8	144	112
meerut	4	199	52
nagpur	4	73	76
lucknow	2	47	42
pune	4 (2 usable)	29	34
kanpur	2	29	28
jaipur	2	22	22
bhopal / agra / patna / indore	2 each	9–12	10–16

18 files, 336 KB in total (Surat: no metro in OSM yet). The bundle is a safety net, not the main source; add any city in the world to `CITIES` in the script and re-run before a release.

5.3 `MetroRoute`

Polyline with cumulative distances; `project(lat, lon)` (nearest segment, equirectangular, O(n)); `pointAt(d)`, `bearingAt(d, dir)` (O(log n)); stations projected onto the line (dropped if > 400 m away; duplicates < 60 m merged); `nextStation / previousStation / nearestStation`.

6. Positioning engine (`UndergroundNavigator`)

6.1 State

Position (`lat, lon, source, uncertainty`), GNSS state, motion (`speed, cruiseSpeed, cruiseEvidence, heading, headingKnown, inVehicle`), track (`route, onTrack, navDist, direction, directionKnown, heldAt, snappedAtStation, lastStopStation`), stop/departure verification (`dvHistory, velE/velN/velPeak/velKnown, motionConfirmed`), compass offset (unit-vector average), radio (`radioMap, trackHmm, belief sync, radioPassedStation`), PDR (`pdrSteps`).

6.2 GNSS and network input

- Validation: accuracy in (0, 2000] m, finite, in range, not (0, 0).
- GNSS with accuracy ≤ 50 m: jump filter (implied speed after subtracting both accuracies ≤ 60 m/s; 3 consecutive rejections reset the reference); after an outage the first fix must be ≤ 15 m or confirmed by a second consistent fix; drift of the estimate is logged.

- Learns cruise speed (EWMA 0.1 of fixes > 4 m/s; each counts as evidence), vehicle state (≥ 5 m/s in vehicle; < 2.5 m/s not), heading, compass offset (see 6.6).
- Weak fixes (network, poor GNSS, **radio fixes**): inflated ×1.5; on a line → 1-D Kalman on distance along track with a 3-σ gate and an off-track tolerance; off a line → 2-D blend. Source becomes NET or RADIO when the fix dominates (gain ≥ 0.5).

6.3 GPS lost

On a line: lock within 250 m; if slow (< 2 m/s) start exactly at the nearest station (≤ 250 m) and watch for the departure; else carry the gap (≤ 15 s) at the last speed. Off a line: carry the gap only if the heading is known.

6.4 Motion update (10 Hz, only without GPS)

```
sample.accel −= learnt bias                          (orientation known)
if orientationKnown:
    record Δv; integrate velocity since the last accepted stop (also during a "glide");
    record (run time, velE, velN) for the odometer
    if phone reads STILL and still not accepted:
        braked = velKnown ? |v| ≤ max(3 m/s, 0.35·peak, 0.04·t_run)  (or peak < 5 m/s)
                          : max suffix |ΣΔv| over 25 s ≥ 0.5·speedAtStill
        accept if braked or still ≥ 15 s (then close the odometer piece, v := 0)
        otherwise treat the sample as MOVING (glide)
STATIONARY: speed = 0 (ZUPT); learn bias after 2 s (τ 30 s); after 5 s → handleStop
WALKING:    on a line → train does not move; off a line → PDR
MOVING:     if unconfirmed after a stop: need |ΣΔv(6 s)| ≥ 1.5 m/s and consistency ≥ 0.5
            on a line: speed += a·û(bearing + compass offset)·dt;
                       speed += (runCruise ?: cruise − speed)·dt/τ;
                       fuse inertial speed (6.12)
            no line, in vehicle: same along the heading; otherwise speed decays
update run cruise (ride memory), curve matching (6.15)
advance along track (hold at next station) or move along heading
uncertainty += dt · (fused speed σ  or  fraction·speed) + 0.3 m/s
```

6.5 Station hold, stops and signal stops

- **Hold:** the estimate may not pass the next station until the train stops there, unless it overshoots by `max(150 m, min(0.5·segment, 2.5σ))`. The station just left and a station the radio saw the train pass are never hold targets.
- **handleStop** chooses the station, in order: (1) the station radio placed the train at just before the stop (≤ 250 m); (2) the **odometer** (6.12): the station at the drift-corrected inertial distance from the last trusted station, within max(80 m, 10 %), else a signal stop; (3) without odometer: next station within `max(400 m, 0.6·segment) + overshoot` or previous within 250 m, accepted if `0.85·N(d; σ) ≥ 0.15/segment`.
- On accept: snap, σ ≤ 40 m, finish the run (ride memory + tunnel learning, 6.13–6.14), learn cruise, set the curve reference. The snap is trusted if σ before it ≤ 300 m or it was placed by the odometer or radio.

6.6 Departure, direction, interchange, compass offset

- Departure window 4 s after the last still moment; strong if mean |a| ≥ 0.25 m/s² **and** consistency |Σa| / Σ|a| ≥ 0.5.
- Bearing corrected by the learnt compass offset. Unknown direction → set from the bearing; known direction → flipped only if within 45° of the opposite direction.
- Interchange: among lines within 300 m, the one whose bearing best matches (margin 20°).

- Compass offset: unit-vector average of (measured – true) bearing, learnt at confirmed departures (rate 0.3) and while GPS shows speeding up / braking ≥ 0.4 m/s² (rate 0.02); used once its confidence ≥ 0.5.

6.7 Radio (`onRadioScan`)

```
no position yet        → radioMap.locate → weak fix (RADIO)
GPS good (≤ 20 m, ≤ 3 s) → learn at GPS position (σ = accuracy)
no line / off line     → radioMap.locate → weak fix (RADIO)
snapped at a station   → learn (if trusted) unless best match ≥ 0.5 weight is > 300 m away (cor
otherwise / conflict   → buffer the scan for tunnel learning; HMM update with L(d)^0.35
```

Likelihood over distance along the line: `L(d) = [0.02 + Σ w_i·N(d; a_i, σ_i+25)] × Π_beacons [0.02 + N(d; a_b, σ_b+15)] × Π_RTT [0.02 + N(|p(d) − ap| − r; √(σ_r² + σ_ap² + 4))]` (absolute floor: weak matches only nudge). The exponent 0.35 tempers correlated consecutive scans. **Kinematic constraint:** once the train has left a station in a known direction, `L(d) = 0` behind it (–40 m margin). HMM: reset to `N(navDist, σ)` when out of sync, else predict with the navigator's shift and `√(σ² − σ_prev²)` blur; update; estimate. Apply only if mode mass ≥ 0.6 and (σ_post < 0.7·σ or the estimate is > 3σ away); backward moves while riding only beyond 3·max(σ_post, σ); posterior σ ≥ 25 m. A station at the corrected spot (≤ 80 m) is marked passed if moving.

6.8 `TrackHmm`

Cells of 10 m over the line. `predict(shift, σ)`: fractional shift with end clamping, Gaussian blur (kernel ≤ ±300 cells, renormalised at the ends), 2 % uniform mix. `update(L)`: multiply, normalise, reject if the total is 0/NaN. `estimate()`: peak, grow down both slopes while ≥ 5 % of the peak and not rising after a valley (< 30 % of peak); mean, σ (incl. cell quantisation) and mass of that mode.

6.9 `RadioMap`

- Usable signals: –95 ≤ RSSI < 0, keys without `= ; tab newline`.
- `learn`: merge into the nearest fingerprint within 20 m (position weighted by samples/σ²; RSSI EWMA with α = 1/min(n+1, 10)); keep the 30 strongest transmitters; anchor = position of the strongest sighting (+3 dB) or a sharper one at similar RSSI; ≤ 5000 fingerprints, the least-sampled evicted (orphan anchors removed).
- `match`: candidates via inverted index; RMS RSSI difference over common transmitters **plus each transmitter that was ≥ –80 dBm at the place but is missing now (counted as –95 dBm; only for radio types this scan listened to)**; `w = exp(−rms²/2·8²)·common/min(|live|,|fp|)`; ≥ 2 common Wi-Fi or ≥ 1 beacon; w ≥ 0.05.
- `locate`: loud beacon (≥ –70 dBm) → anchor (σ + 15 m); else weighted centroid of the best 3 matches with w ≥ 0.2 (accuracy = spread + mean σ + 25 m); the more accurate wins.
- File `files/radio_map.txt`: `# radio map v1`, `F\tlat\tlon\tσ\tn\tkey=rssi;…`, `A\tkey\tlat\tlon\tσ\trssi`; malformed lines skipped.

6.10 PDR (`Pdr`)

`stepLength = clamp(0.3 + 0.2·cadence, 0.45, 0.9)` (0.7 m when unknown). Heading: device top edge when the phone is flat, camera direction when upright; null when neither is horizontal.

6.12 Inertial speed, bias and odometer

- **Bias:** during accepted stillness (≥ 2 s) `bias += min(1, dt/30 s)·(a_raw − bias)`, kept across stops; subtracted from every sample.
- **Velocity:** integrated from the last accepted stop (v = 0). Fusion with the model speed: `σ_meas = 0.5 + 0.1·t_since_stop`, `σ_model = f·max(speed, cruise)` (f = 0.12 learnt run, 0.30 otherwise); `w = σ_model²/(σ_model² + σ_meas²)`; `speed += w·(|v| − speed)·min(1, dt/1 s)`; fused σ = `σ_model·σ_meas/√(σ_model² + σ_meas²)` drives the uncertainty growth.
- **Odometer:** the run's velocity samples are split into pieces at each zero-velocity update; in each piece the residual velocity at its end is removed linearly in time; the distance is `∫|v_corrected| dt`. Used at stops (6.5) and for placing tunnel scans (6.14).

6.13 Ride memory (`RideMemory`)

`record(route, from, to, meters, movingSeconds)` for adjacent trusted stops (average 3–35 m/s), EWMA of the moving time (α = 1/min(n+1, 5)). `cruise()` solves `D = v·T − v²·(1/2a + 1/2b)` with a = b = 1 m/s² (triangular peak `√(D/k)` if the run never reaches cruise), clamped 5–30 m/s. File `ride_memory.txt` (`# ride memory v2`, `S\tkey\tmeters\tseconds\tn`).

6.14 Tunnel radio learning

Scans heard between two trusted stops are buffered (≤ 400) with their run time. At the arrival, each is placed at `start + dir·frac·D`, where `frac` is its odometer distance / total (or model distance / total without orientation), and learnt with σ = 15 m + 0.04·min(frac, 1–frac)·D (0.10 without the odometer). Runs whose odometer or model distance is outside 0.6–1.6 × the track distance are not learnt.

6.15 Curve matching

Gyroscope turn since the last station snap (reference), integrated while the phone rests (|ratel ≤ 15 °/s, not walking; otherwise invalid until the next station). After a turn of ≥ 15° has settled (2 s-smoothed rate < 0.5 °/s for 3 s): σθ = 4° + 10 % of the turn + 0.05 °/s · time. If `|turnBetween(ref, navDist) − turn| ≤ 2σθ` nothing is done; otherwise the position moves to the nearest point ahead of the reference within ±3σ that agrees (backward only beyond 3σ). `MetroRoute.turnBetween` sums signed segment-bearing changes (segments < 2 m and vertex turns > 100° treated as mapping noise).

6.16 Weak satellite speed

A GNSS fix too poor for position (≤ 100 m accuracy) with a speed: `speed += 0.3·(gnss − speed)`.

6.11 Output

`NavState` adds `radioCorrections`, `radioPlacesLearnt`, `radioSignalsHeard`, `pdrSteps`, `curveCorrections`, `segmentsLearnt` to the previous fields; `PositionSource` adds `RADIO`. Status texts include "Wi-Fi/BT-assisted", "Wi-Fi/Bluetooth location" and "step counting". Confidence: ≤ 30 m HIGH, ≤ 100 MEDIUM, ≤ 300 LOW, else VERY_LOW.

7. Error handling

Failure	Handling
Invalid / stale fixes	rejected (validation, 30 s staleness, jump filter)
Sensor missing	capability flags; feature disabled; UI warning
Radio permission / hardware missing, Bluetooth or Wi-Fi off	that part skipped; UI warning with a settings shortcut
Throttled Wi-Fi scan	stale list detected by timestamp and ignored
RTT failure	callback resets the busy flag; next tick retries
Corrupt radio map / cache / bundle file	skipped with a log; new data rebuilt
Disk full / crash during save	temp file + rename; old file intact
Overpass down / slow	bundle and cache used; back-off; retry on network
Wrong station snap	not learnt unless trusted; radio conflict blocks learning and can relocate

8. Permissions (manifest)

`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`, `ACTIVITY_RECOGNITION`, `INTERNET`, `ACCESS_NETWORK_STATE`, `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION`, `POST_NOTIFICATIONS`, `WAKE_LOCK`, `ACCESS_WIFI_STATE`, `CHANGE_WIFI_STATE`, `BLUETOOTH`/`BLUETOOTH_ADMIN` (≤ API 30), `BLUETOOTH_SCAN` (without `neverForLocation`, because beacons are used for location), `NEARBY_WIFI_DEVICES`, `WRITE_EXTERNAL_STORAGE` (≤ API 28). Optional features: `wifi`, `wifi.rtt`, `bluetooth_le`, `location.gps`, `sensor.stepdetector`.

9. Configuration reference

9.1 Motion and stops

Constant	Value	Meaning
DEFAULT / MIN / MAX_CRUISE_MPS	12 / 5 / 30	cruise speed prior and limits
CRUISE_PULL_TAU_SEC (uncertain)	20 (6) s	pull of speed toward cruise
MAX_ACCEL_MPS2	2.5	clamp on measured acceleration
STATION_STOP_MS	5 s	still time before a stop is handled
STOP_CHECK_MIN_MPS	5	below this no braking proof needed
STOP_RESIDUAL_MPS / _FRACTION	3 / 0.35	integrated velocity allowed after a real stop
BRAKE_MIN_FRACTION	0.5	braking Δv needed when velocity is not known
LONG_STILL_ACCEPT_MS	15 s	still this long = stop regardless

Constant	Value	Meaning
DEPARTURE_CONFIRM_DV_MPS / WINDOW	1.5 / 6 s	departure evidence
DEPARTURE_MIN_CONSISTENCY	0.5	one-directionality of the push
UNCONFIRMED_MOVING_ACCEPT_SEC	20	fallback for a misdetected stop
DEPARTURE_WINDOW_SEC / MIN_ACCEL	4 s / 0.25	direction detection
DIRECTION_FLIP_MARGIN_DEG	45	known direction flips only near-opposite
STATION_STOP_PRIOR	0.85	station vs. signal stop
STOP_SNAP_AHEAD_MIN / FRACTION, BEHIND	400 m / 0.6, 250 m	snap windows
MIN_OVERSHOOT_METERS / OVERSHOOT_FRACTION	150 / 0.5	release of a station hold
TRACK_SPEED_ERROR_FRACTION (unlearnt)	0.15 (0.5)	uncertainty growth
CRUISE_EVIDENCE_NEEDED	10	evidence for a learnt cruise
COMPASS_LEARN_RATE_GPS / _DEPARTURE	0.02 / 0.3	compass-offset learning

9.3 Inertial speed, odometer, learning, curves

Constant	Value	Meaning
BIAS_LEARN_AFTER_STILL_MS / TAU	2 s / 30 s	accelerometer bias learning
VEL_MEAS_BASE / GROWTH	0.5 m/s / 0.1 m/s per s	inertial speed error
MODEL_SPEED_ERROR_FRACTION / SEGMENT	0.30 / 0.12	model speed error (unknown / learnt run)
STOP_RESIDUAL_GROWTH_MPS_PER_SEC	0.04	stop check allowance per second of run
ODOMETER_MIN_TOLERANCE / FRACTION	80 m / 0.10	odometer station match
MIN_RUN_FOR_ODOMETER_METERS	200	shorter runs use the fallback
HOLD_SIGMAS	2.5	hold at most 2.5σ past a station
TUNNEL_LEARN_BASE / FRACTION (inertial)	15 m / 0.10 (0.04)	tunnel fingerprint σ
CURVE_MIN_TURN / AGREE_SIGMAS	15° / 2	curve matching
HANDLING_YAW_DPS	15 °/s	phone turned in the hand
RADIO_EVIDENCE_EXPONENT / MIN_SHARPENING	0.35 / 0.7	radio tempering / gain needed
DEPARTED_STATION_MARGIN_METERS	40	kinematic constraint
MISSING_LOUD_RSSI	–80 dBm	absence evidence

9.2 Radio and HMM

Constant	Value	Meaning
RADIO_LEARN_GPS_ACCURACY_METERS	20	learn only with good GPS
RADIO_LEARN_MAX_SNAP_UNCERTAINTY_METERS	300	trusted station snap
RADIO_CONFLICT_METERS / MIN_WEIGHT	300 / 0.5	radio contradicts the snap
RADIO_LIKELIHOOD_FLOOR	0.02	no single reading is decisive
RADIO_MIN_MODE_MASS	0.6	apply only unambiguous beliefs
RADIO_PASS_RADIUS_METERS / STOP_CONFIRM	80 / 250	passed-station and stop confirmation
SIGNAL_SIGMA_DB	8	RSSI matching tolerance
MERGE_RADIUS_METERS	20	one fingerprint per place
BLE_NEAR_RSSI	–70 dBm	beacon checkpoint
FINGERPRINT_SPREAD_METERS	25	spatial spread of a fingerprint
TrackHmm cell / teleport	10 m / 0.02	grid and recovery
WIFI_SCAN_INTERVAL_MS / WIFI_MAX_AGE_MS	30 s / 10 s	Wi-Fi scanning
BLE_WINDOW_MS / EMIT_INTERVAL_MS	3 s / 2 s	BLE aggregation

10. Tests (82, all passing)

Class	Tests	Covers
`UndergroundNavigatorTest`	11	track lock, stop snapping, hold, walking, departure, destination, DR, recovery, jump filter
`EdgeCaseTest`	15	invalid fixes, network gating/outliers, cold start, gap DR, recovery confirmation, cruise learning, interchange, uncertainty, mock
`MetroRouteTest`	7	projection, stations, stitching
`RideSimulationTest`	6	straight-line rides 80/108 km/h, boarding underground, 60° compass, network, 20 seeds
`UndergroundScenarioTest`	20	20 km curved tunnel; slow/fast; 12 seeds × compass errors; reverse boarding; false stills (with/without checks); jiggle; signal stop; express; radio learning; cold start with radio only; no line data; **realistic line (per-run speeds, 6 seeds); express on it (odometer); learnt runs + radio on express; second ride; no orientation sensor; curve matching idle when consistent and correcting a contradicting estimate; phone turned at platforms**
`PositioningComponentsTest`	22	RadioMap, TrackHmm, navigator radio learning / cold start, PDR, beacon parsing, **RideMemory, run cruise, line curves**
`BundledLinesTest`	1	every bundled city parses with the app parser

`TunnelSimulator` models the curved line, train kinematics (optionally a fixed cruise per run), IMU (tangential + centripetal, compass error, bias, noise), gyroscope (turn rate + noise + bias; fast rotations while jiggled), GPS until the portal, the 6 s watchdog, false stills, jiggle, express, signal stops, and a radio world; it can carry the radio map and ride memory from one ride to the next.

11. Defects found by the simulations and fixed

Found	Cause	Fix
Smooth cruise read as "still" collapsed the ride (errors up to 13 km)	stop accepted without braking evidence	integrated-velocity stop verification
Phone jiggle at platforms reversed the direction	jiggle averaged to a "departure"	directional-consistency test for departures and confirmations
Real station stop taken for a signal stop	uncertainty too small while cruise unlearnt	honest unlearnt speed error
Direction flipped mid-line with a 60° compass error	any disagreeing departure flipped	learnt compass offset; flip only when clearly opposite
Radio froze the display after leaving a station (≈ 750 m)	tiny backward correction re-armed the hold	monotonic correction; never hold at the station just left / passed
Arrival marked as "passed", stop called a signal stop	passing marker at arrival	radio evidence confirms the stop
Bimodal HMM belief reported as its midpoint	mode grew across the floor	edge 5 % + valley stop
Gap dead-reckoned toward north with unknown heading	default heading 0°	only with a known heading
First-ride radio doubled the error	weak matches as decisive as strong; missing loud signals ignored; correlated scans	absolute floor, absence evidence, tempering (0.35), clear-gain rule
Radio pulled a departing train back behind the station	radio before/after a station is symmetric	kinematic constraint behind the station just left
On a line with different run speeds, a stop at a fast run was called a signal stop → off by one	cruise pull + over-confident error	inertial speed with bias removal; honest 30 % run-speed error
Express/signal stops identified from a drifting estimate	estimate lags/leads after holds	odometer at stops
Segment "scale" memory double-corrected the new inertial speed	two corrections of the same error	memory stores run time → run cruise
Tunnel fingerprints placed 50–100 m off	model error not proportional along a run	ZUPT-smoothed inertial placement
Curve evidence worsened worst cases	one-sided evidence shifted a consistent estimate	consistency check, move only on contradiction
Real stop rejected after jiggling the phone	5 s bias averaging too noisy	30 s averaging across stops; allowance grows with run time
Odometer lost 88 m per false "still"	glide samples not integrated	integrate during glides
Running-odometer hold release caused false "passed"	uncorrected running distance over-reads	removed; odometer used only at stops

12. Field validation and tuning

1. Log two end-to-end rides per line (the first teaches the radio map).
2. From the log: stop events vs. timetable; "GPS recovered … off by N m"; radio corrections.
3. Symptoms → constants: stops missed → `STATION_STOP_MS`, `STILL_ACCEPT` thresholds in `SensorHub`; stops at wrong station → `STATION_STOP_PRIOR`, snap windows; lag/lead between stations → cruise constants; wrong direction → `DEPARTURE_MIN_ACCEL`, consistency; radio jumps → `RADIO_LIKELIHOOD_FLOOR`, `RADIO_MIN_MODE_MASS`, `SIGNAL_SIGMA_DB`.
4. After any change: `./gradlew :app:testDebugUnitTest` must stay green.

13. Known limitations

- Radio and run-speed learning need one ride per line (no shared database, by design).
- Express trains on a never-ridden line: the display waits at a skipped station until the overshoot allowance (up to ~800 m lag in simulation); the next stop is always identified correctly and one learning ride removes the lag.
- RTT depends on 802.11mc access points (rare in metros today).
- Wi-Fi scan throttling (4 per 2 min) limits Wi-Fi to stations; BLE fills the gaps.
- Curve matching adds nothing measurable when inertial speed works; it is a safety net.
- The barometer is not used as a station cue (piston-effect pressure swings would add false stops).
- Without line data, accuracy is that of dead reckoning (~7 % of distance in simulation).
- All accuracy figures are from simulation; real phones' bias, tilt and noise must be checked in the field (section 12).